

DEPARTEMENT MATHÉMATIQUES ET INFORMATIQUE

**Rapport de
Projet de Mobile
Quail.ma**

Réalisé par :
RAHIOUI Youssef
NAZIH Youssef
HATTABI Youness

Encadrant :
Pr. Hassna BENSAG

Année universitaire : 2026 - 2027

Contents

1	Introduction	2
1.1	Contexte et motivation	2
1.2	Problématique	2
1.3	Solution proposée	2
2	Conception	4
2.1	Diagramme de classes	4
2.2	Diagramme de cas d'utilisation	5
3	Implémentation	6
3.1	Architecture système	6
3.2	Outils et technologies utilisés	6
3.2.1	Environnements de développement (IDEs)	7
3.2.2	Langages et frameworks	7
3.2.3	Packages et dépendances Flutter	7
3.3	Modèles de données et API	8
3.3.1	Endpoints principaux	8
3.3.2	Flux de statut des commandes	8
3.4	Pages et widgets de l'application	8
3.4.1	Interface d'administration	8
3.4.2	Interface client — Boutique et catalogue	10
3.4.3	Interface client — Panier et commandes	11
3.5	Gestion de l'état avec GetX	11
3.6	Persistance et sécurité	12
4	Conclusion	13

Chapter 1

Introduction

1.1 Contexte et motivation

Ce projet s'inscrit dans le cadre de notre formation en GLSID (Génie du Logiciel et des Systèmes Informatiques Distribués) à l'ENSET Mohammedia. En tant qu'étudiants en deuxième année, ce travail représente le projet final du module de développement mobile. Il nous permet de mettre en pratique l'ensemble des compétences acquises au cours de notre cursus : conception d'applications, programmation orientée objet, gestion de bases de données, et architecture client-serveur.

Notre équipe, composée de trois étudiants, a souhaité réaliser une application complète, depuis l'analyse du besoin jusqu'à la réalisation technique, pour répondre à un problème concret dans le contexte marocain. Avec **quail.ma**, nous avons cherché à démontrer notre capacité à concevoir et développer une solution logicielle full-stack, moderne et fonctionnelle.

1.2 Problématique

Au Maroc, la filière des cailles et des œufs de caille demeure un marché de niche, mal structuré et peu visible pour le grand public. Les éleveurs, souvent de petite ou moyenne taille, peinent à écouler leur production faute de canaux de distribution adaptés. Parallèlement, les consommateurs intéressés par ces produits — particuliers, restaurateurs, ou amateurs de produits du terroir — éprouvent des difficultés à trouver des fournisseurs fiables, à comparer les prix et à se faire livrer dans des délais raisonnables.

L'absence d'une plateforme dédiée crée ainsi une rupture significative entre l'offre et la demande, ce qui freine le développement de ce secteur pourtant doté d'un fort potentiel nutritionnel et économique. Les transactions restent largement informelles, la traçabilité est quasi inexistante, et la digitalisation du commerce de ces produits est quasiment nulle.

1.3 Solution proposée

Pour répondre à cette problématique, nous proposons **quail.ma**, une application mobile et desktop qui met directement en relation les éleveurs de cailles et les consommateurs marocains.

L'application offre deux expériences utilisateur distinctes :

- **Côté client** : parcourir un catalogue structuré de produits (œufs frais, cailles vivantes, couples de cailles), passer commande en quelques clics, suivre l'état de la livraison en temps réel, et consulter l'historique des commandes.

- **Côté administrateur** : gérer les stocks de produits, traiter les commandes reçues, mettre à jour les statuts de livraison, et administrer le catalogue via une interface dédiée.

`quail.ma` vise ainsi à digitaliser et dynamiser une filière encore largement artisanale, tout en garantissant transparence, traçabilité et accessibilité aux clients. Le projet adopte une architecture client-serveur moderne avec une application frontale développée en *Flutter* et une API backend en *Go*.

Chapter 2

Conception

Pour bien définir la structure et le fonctionnement de notre application, nous avons réalisé deux diagrammes UML essentiels. Le diagramme de classes présente l'organisation des données et les relations entre les différentes entités du système, tandis que le diagramme de cas d'utilisation décrit les interactions entre les utilisateurs et l'application.

2.1 Diagramme de classes

Le diagramme de classes suivant modélise les entités métier de l'application `quail.ma`. Il repose sur une architecture simple mais robuste, adaptée à la gestion d'un catalogue de produits et au suivi des commandes.

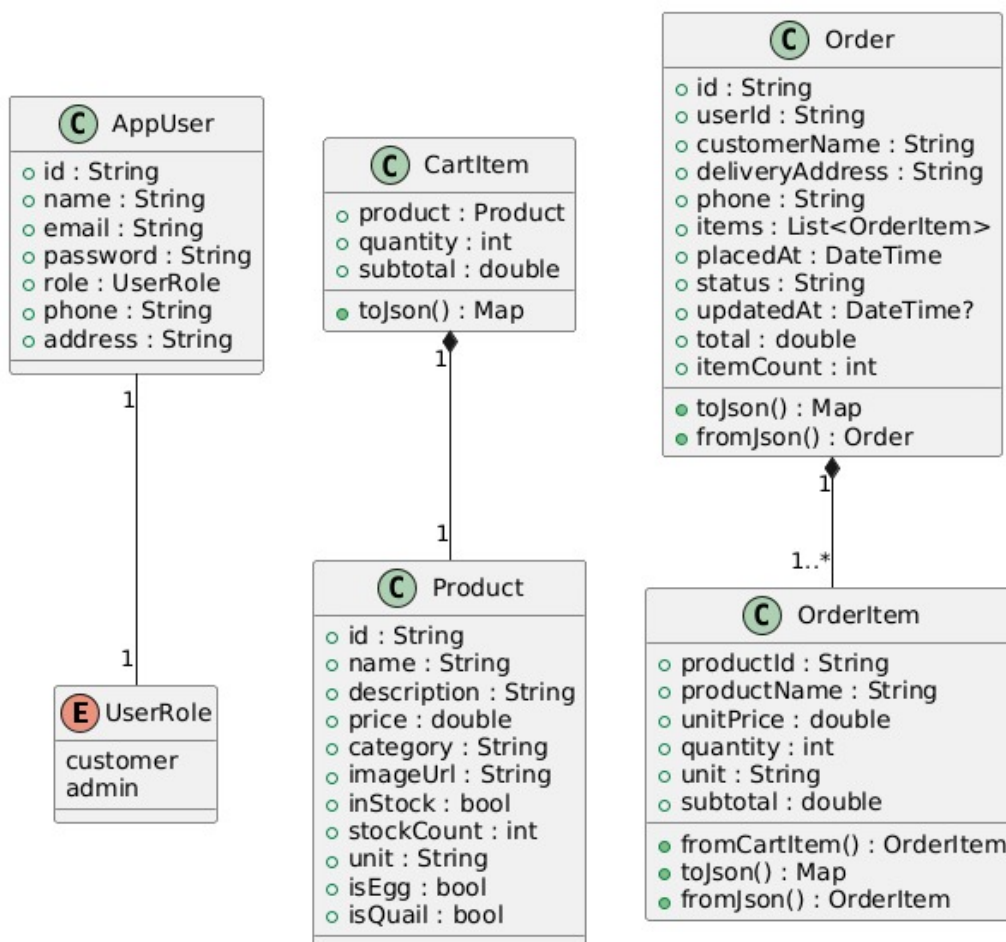


Figure 2.1: Diagramme de classes du système `quail.ma`

Remarques sur la conception :

- La classe `OrderItem` utilise une *dénormalisation* des données (nom, prix, unité au moment de la commande) afin de préserver l'historique exact des transactions, même si le produit est modifié ultérieurement.
- La classe `AppUser` distingue deux rôles via un énuméré `UserRole` : `admin` et `customer`.
- Le mot de passe est stocké côté backend mais jamais exposé dans les réponses API.

2.2 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation suivant illustre les fonctionnalités principales offertes par l'application, selon le type d'acteur.

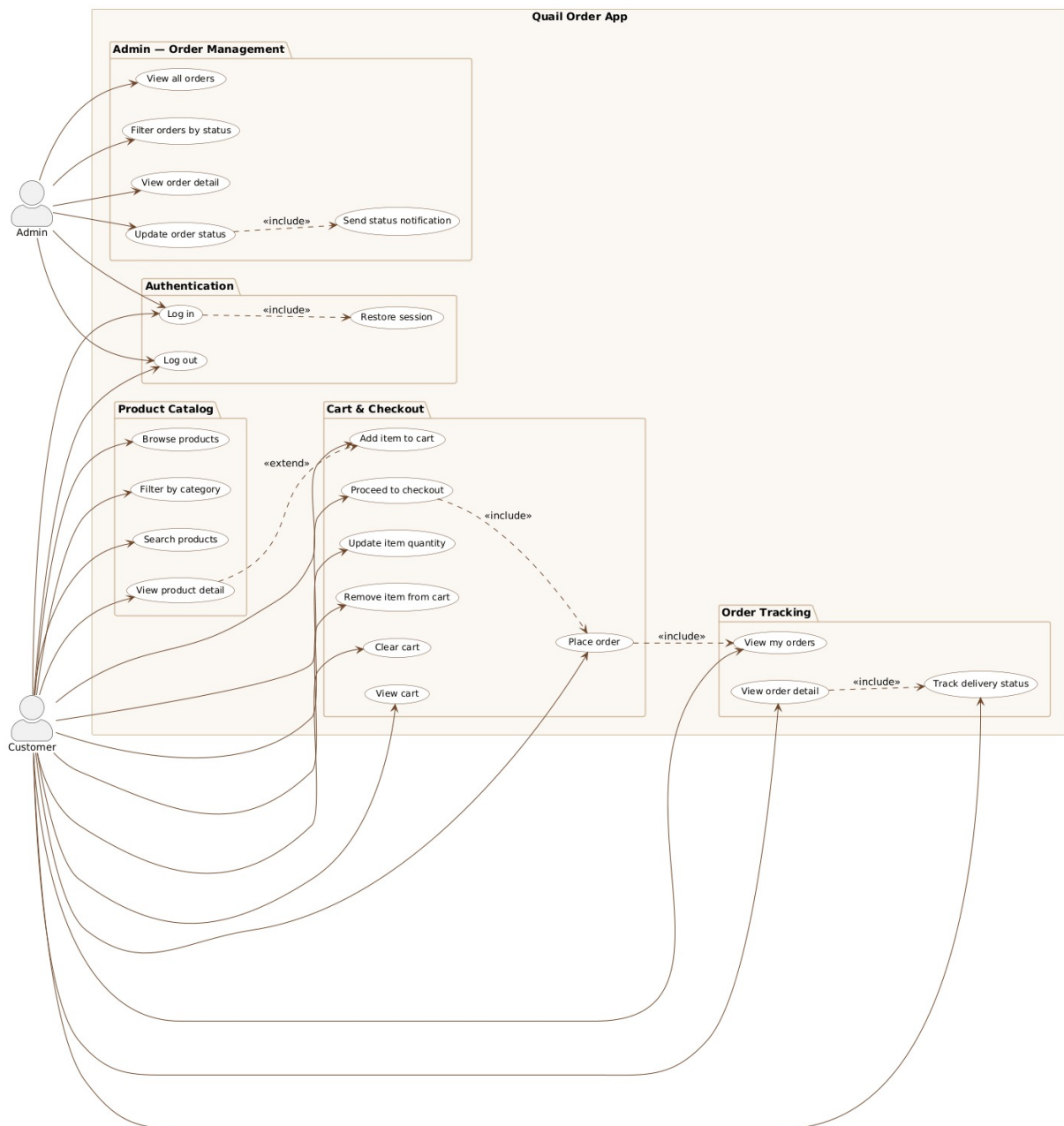


Figure 2.2: Diagramme de cas d'utilisation du système quail.ma

Chapter 3

Implémentation

3.1 Architecture système

L'application `quail.ma` adopte une architecture client-serveur classique mais moderne. La communication entre le client Flutter et le serveur Go s'effectue via HTTP avec des payloads JSON sur le port 8080.

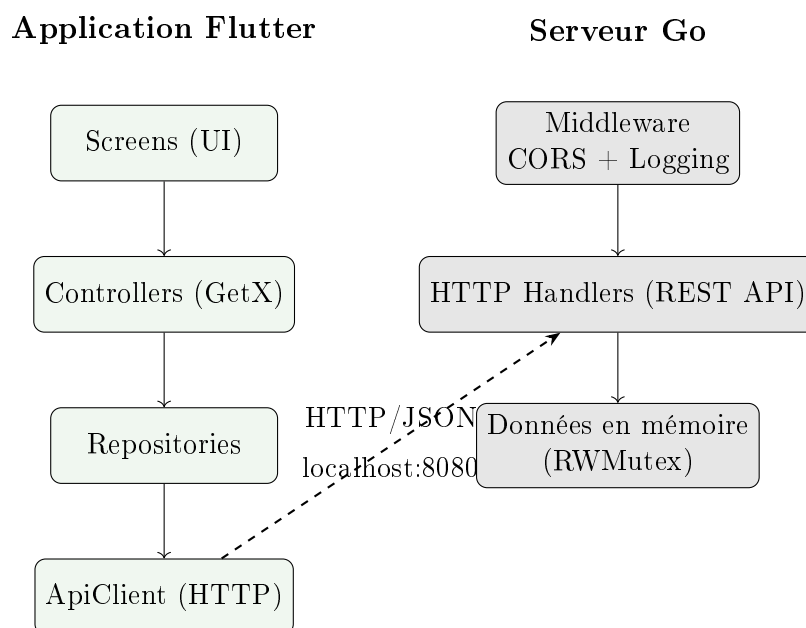


Figure 3.1: Architecture client-serveur de `quail.ma`

3.2 Outils et technologies utilisés

Pour réaliser notre application, nous avons utilisé plusieurs outils et technologies, répartis selon leur rôle dans le projet.

3.2.1 Environnements de développement (IDEs)

IDE	Utilisation	Justification
Android Studio	Application Mobile	Propose de nombreux plugins pour le développement mobile, un émulateur Android intégré et un excellent support pour Flutter.
VS Code	Backend	Léger et rapide, avec une extension Go qui offre l'autocomplétion, le formatage et le débogage.

Table 3.1: IDEs utilisés dans le projet

3.2.2 Langages et frameworks

Langage/Framework	Utilisation	Justification
Flutter avec GetX	Application Mobile	Flutter permet de créer une interface moderne avec un seul code source multiplateforme. GetX simplifie la gestion d'état réactive et la navigation.
Go (Golang)	Backend	Langage compilé, rapide et simple, bien adapté à la création d'APIs REST performantes. L'utilisation de la bibliothèque standard garantit la légèreté.

Table 3.2: Langages et frameworks utilisés

3.2.3 Packages et dépendances Flutter

L'application frontale intègre plusieurs packages essentiels :

- **http** (^1.2.2) : client HTTP pour la communication avec l'API.
- **get** : gestion d'état réactive (Rx), injection de dépendances et navigation.
- **cached_network_image** : mise en cache des images produits.
- **flutter_local_notifications** : notifications locales lors des changements de statut de commande.
- **shared_preferences** : persistance de la session utilisateur.
- **font_awesome_flutter** : icônes riches pour l'interface.
- **intl** : formatage des dates et localisation.

3.3 Modèles de données et API

Le backend Go expose une API REST complète. Les données sont stockées en mémoire avec une protection par `sync.RWMutex` pour garantir la thread-safety.

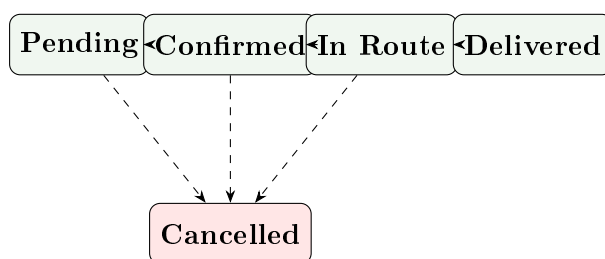
3.3.1 Endpoints principaux

Méthode	Chemin	Description
GET	/api/products	Liste des produits (filtrage par catégorie possible)
POST	/api/products	Création d'un produit (admin)
GET	/api/products/{id}	Détail d'un produit
PUT	/api/products/{id}	Mise à jour d'un produit (admin)
DELETE	/api/products/{id}	Suppression d'un produit (admin)
POST	/api/auth/login	Authentification par email/mot de passe
GET	/api/users/{id}	Profil utilisateur (sans mot de passe)
GET	/api/orders	Liste des commandes (filtrage par utilisateur/statut)
GET	/api/orders/{id}	Détail d'une commande
POST	/api/orders	Passation d'une nouvelle commande
PATCH	/api/orders/{id}/status	Mise à jour du statut (admin)

Table 3.3: Endpoints de l'API REST

3.3.2 Flux de statut des commandes

Les commandes suivent un cycle de vie strict :



3.4 Pages et widgets de l'application

Cette section présente les principales interfaces de l'application, illustrées par des captures d'écran.

3.4.1 Interface d'administration

L'interface admin est accessible après authentification avec le compte administrateur. Elle offre une gestion complète via deux onglets principaux : **Commandes** et **Produits**.

PRODUCT	PRICE	STOCK
Live Japanese Quail Quail per bird	35.00 MAD	40
Laying Quail Hen Quail per bird	50.00 MAD	20
Quail Pair (1M + 1F) Quail per pair	90.00 MAD	10
Fresh Quail Eggs - Dozen Eggs per dozen (12)	12.00 MAD	200
Fresh Quail Eggs - Tray Eggs per tray (30)	28.00 MAD	150
Hatching Eggs - Dozen Eggs per dozen (12)	20.00 MAD	0

Figure 3.2: Panneau d'administration — Gestion des produits

La figure 3.2 présente l'onglet *Produits* du panneau d'administration. On y retrouve la liste complète des produits avec leur image, nom, catégorie, prix et stock. Les actions d'édition et de suppression sont disponibles pour chaque ligne. Le bouton *+ Add Product* permet d'ajouter un nouveau produit.

Order #	Status	Items	Date	Total
Order #171565	Pending	Laying Quail Hen x2	14 May 2026, 08:00	100.00 MAD
Order #171559	Confirmed	Live Japanese Quail x3	13 May 2026, 14:00	105.00 MAD
Order #171550	Delivered	Fresh Quail Eggs - Dozen x2, Fresh Quail Eggs - Tray x1	12 May 2026, 10:30	52.00 MAD

Figure 3.3: Panneau d'administration — Gestion des commandes

La figure 3.3 illustre l'onglet *Commandes*. L'administrateur peut filtrer les commandes par statut (All, Pending, Confirmed, In Route, Delivered, Cancelled) via des puces sélectives. Chaque carte de commande affiche l'identifiant, les articles, la date, le montant total et le statut actuel.

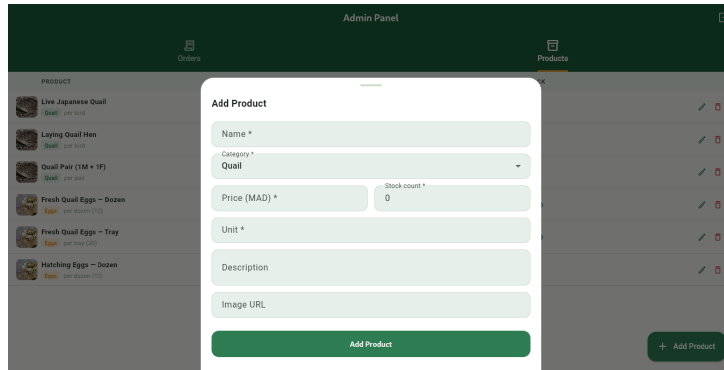
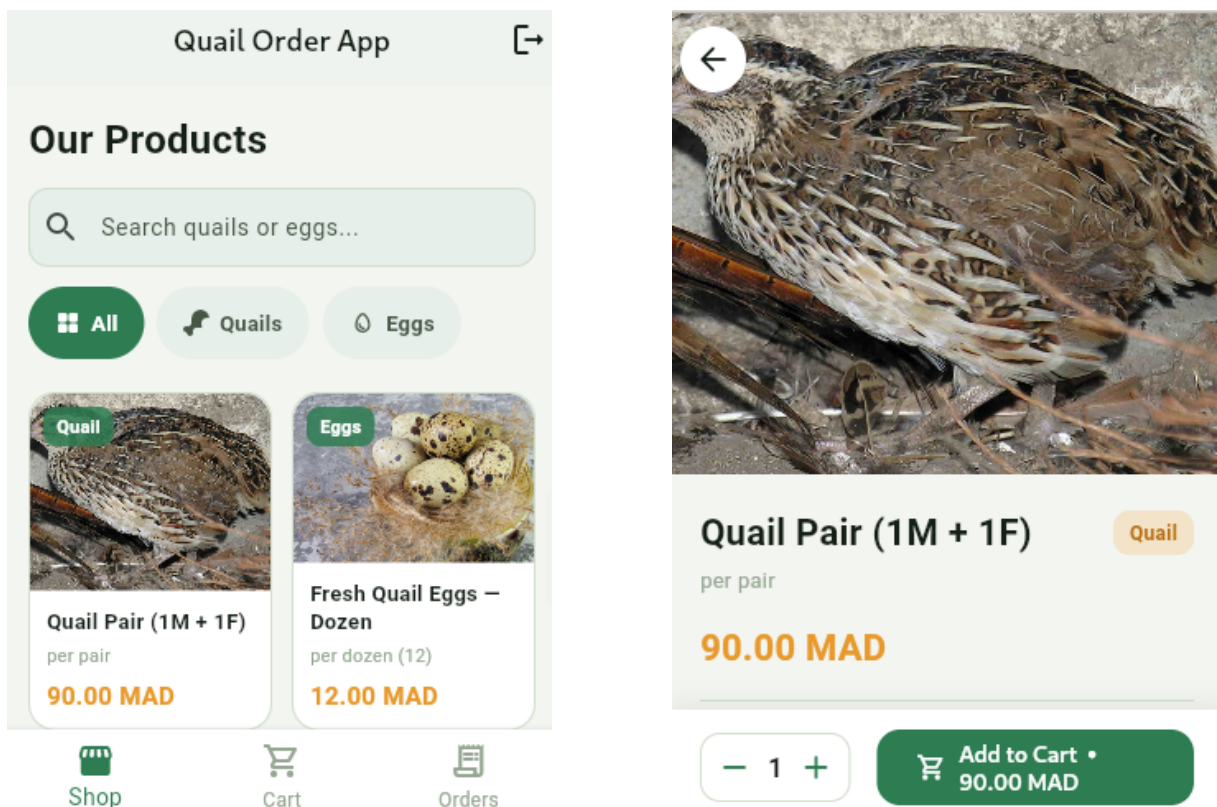


Figure 3.4: Formulaire d'ajout d'un produit (Bottom Sheet)

La figure 3.4 montre le formulaire modal d'ajout de produit. Les champs obligatoires incluent le nom, la catégorie (Quail ou Eggs), le prix en MAD, le stock et l'unité. La description et l'URL de l'image sont optionnelles.

3.4.2 Interface client — Boutique et catalogue

L'expérience client débute par un écran d'accueil présentant le catalogue de produits, avec des fonctionnalités de recherche et de filtrage par catégorie.



(a) Écran d'accueil — Catalogue

(b) Détail d'un produit

Figure 3.5: Interfaces de navigation et de détail produit

La figure 3.5a montre l'écran *Shop* avec la barre de recherche, les filtres par catégorie (All, Quails, Eggs) et la grille de produits. Chaque carte affiche l'image, la catégorie,

le nom, l'unité et le prix. La navigation inférieure permet d'accéder au panier et aux commandes.

La figure 3.5b présente l'écran de détail d'un produit. L'utilisateur peut ajuster la quantité via des boutons +/- et ajouter l'article au panier. Le prix total se met à jour dynamiquement.

3.4.3 Interface client — Panier et commandes

Le panier permet de réviser les articles sélectionnés avant de procéder au paiement. La validation d'une commande déclenche un appel API et décrémente automatiquement les stocks.

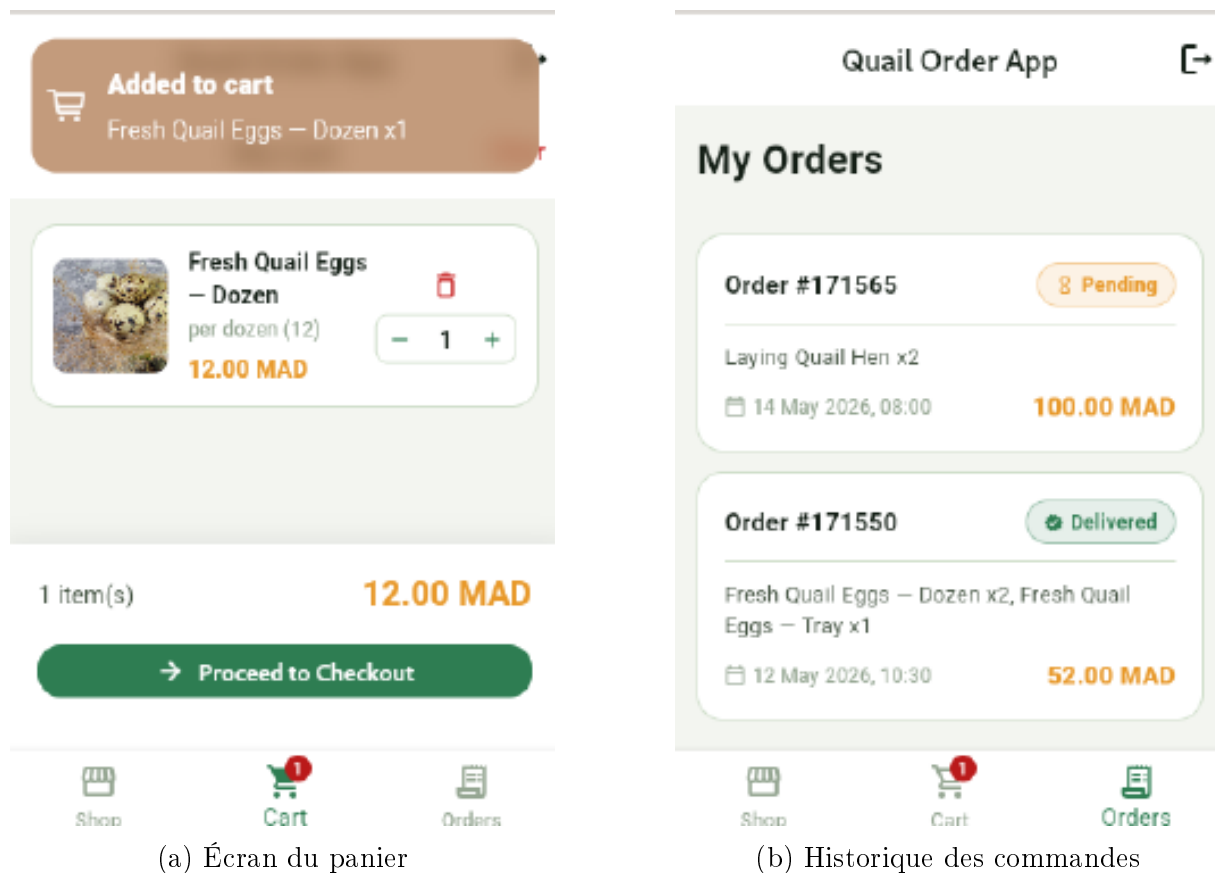


Figure 3.6: Interfaces de panier et suivi des commandes

La figure 3.6a illustre le panier avec une notification de confirmation d'ajout. L'utilisateur peut modifier les quantités, supprimer des articles, voir le nombre total d'articles et le montant global. Le bouton *Proceed to Checkout* finalise la commande.

La figure 3.6b présente l'historique des commandes du client. Chaque commande affiche son identifiant, les articles, la date de passation, le montant total et un badge de statut coloré (Pending, Delivered, etc.).

3.5 Gestion de l'état avec GetX

L'application utilise GetX pour une gestion d'état réactive et efficace. Les contrôleurs maintiennent des observables (`RxList`, `RxBool`) qui déclenchent automatiquement la re-

construction des widgets `Obx` concernés.

Listing 3.1: Exemple de reactive state avec `GetX`

```
class AdminController extends GetxController {  
  final RxList<Order> orders = <Order>[].obs;  
  final RxBool isLoading = false.obs;  
  
  Future<void> loadOrders() async {  
    isLoading.value = true;  
    final result = await OrderRepository().fetchOrders();  
    orders.assignAll(result);  
    isLoading.value = false;  
  }  
}
```

Cette approche évite les appels excessifs à `setState()` et garantit une interface fluide, même lors du chargement de listes de données.

3.6 Persistance et sécurité

- **Session** : Les données utilisateur sont sérialisées en JSON et stockées via `SharedPreferences`, permettant une reconnexion automatique.
- **Sécurité** : Le mot de passe n'est jamais renvoyé au client. Les erreurs API sont normalisées sous la forme `{ "error": "message" }` pour un traitement uniforme côté Flutter.
- **Thread-safety** : Le backend utilise `sync.RWMutex` pour protéger les slices de produits et de commandes contre les conditions de course.

Chapter 4

Conclusion

Le projet **quail.ma** constitue une réalisation complète et aboutie du module de développement mobile de notre formation GLSID. À travers ce projet, nous avons démontré notre capacité à concevoir, architecturer et développer une application full-stack moderne, répondant à un besoin réel du contexte marocain.

Les principales réalisations techniques incluent :

- Une **architecture client-serveur** robuste avec séparation claire des responsabilités.
- Une **API REST** développée en Go, performante et thread-safe.
- Une **interface utilisateur réactive** construite avec Flutter et GetX.
- Une **logique métier complète** incluant la gestion des stocks, le cycle de vie des commandes, et la dénormalisation des données pour préserver l'intégrité historique.

Ce projet nous a permis d'approfondir nos connaissances en développement mobile, en conception d'APIs, et en gestion d'état réactive. Les défis rencontrés — notamment la sérialisation des dates entre Dart et Go, la gestion des hôtes API selon la plateforme, et la prévention des boucles de reconstruction infinies — ont été autant d'occasions d'apprentissage.

quail.ma illustre parfaitement comment la technologie peut servir à la digitalisation de filières traditionnelles marocaines. Il pourrait être étendu à l'avenir par l'ajout d'une vraie base de données persistante, d'un système de paiement en ligne, et d'une géolocalisation des éleveurs pour optimiser les livraisons.

Youssef Nazih, Youssef Rahioui, Youssef Hattabi
Année universitaire 2025/2026